

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

NIVEL 1 (100% PRINCIPIANTES ABSOLUTOS)

Curso 1: Fundamentos Básicos de Python

«De Cero a Programador: Los 4 Pilares Lógicos, Colecciones y Proyecto Integrador»

Manual completo del curso compilado oficialmente. Incluye fundamentación teórica, metáforas conceptuales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor Principal: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Total de Módulos / Clases: 8 | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura del Manual de Curso

Este libro unificado contiene el desglose secuencial de cada módulo formativo, estructurado con rigor de ingeniería y diseñado para servir como manual de referencia permanente.

Plan de Estudios y Hoja de Ruta

El curso está dividido en las siguientes unidades temáticas secuenciales:

Código	Unidad y Metáfora	Objetivo Principal
Clase 01	Clase 01: El Panorama General de la Programación «El Asistente, las Cajas, el Semáforo y la Licuadora»	Comprender que programar es dar instrucciones secuenciales precisas y dominar la...
Clase 02	Clase 02: Variables y Tipos de Datos «El Almacén, el Collar de Letras y el Micrófono»	Comprender la diferencia fundamental entre tipos numéricos y texto, y cómo Pytho...
Clase 03	Clase 03: Control de Flujo - Condicionales «El Guardia de la Puerta y el Menú de Opciones»	Comprender la evaluación de expresiones booleanas y la exclusión mutua en cadena...
Clase 04	Clase 04: Control de Flujo - Bucles «Las Vueltas a la Pista y el Termostato»	Diferenciar con claridad cuándo emplear una iteración acotada (for) vs una itera...
Clase 05	Clase 05: Listas y Colecciones de Datos «La Mochila del Programador y los Casilleros»	Comprender la indexación basada en cero (0-indexed), la mutabilidad de listas y ...
Clase 06	Clase 06: Diccionarios y Mapeos Clave-Valor «La Agenda Telefónica y el Expediente Médico»	Comprender la indexación por clave semántica en lugar de posición numérica y la ...
Clase 07	Clase 07: Funciones Reutilizables y Modulares «El Electrodoméstico y la Entrega del Cajero»	Comprender el principio DRY (Don't Repeat Yourself), la diferencia entre print y...
Clase 08	Clase 08: Integración Total & Proyecto Integrador «El Casco de Seguridad y Salir a Rodar en Bici»	Comprender cómo se interconectan todos los pilares del lenguaje para crear una a...

Criterio de Progresión

Cada unidad construye sobre los cimientos de la anterior. Se recomienda dominar la teoría, analizar el diagrama de arquitectura y replicar el código de cada sección antes de avanzar a la siguiente.

Clase 01: El Panorama General de la Programación

Toda aplicación moderna, desde un script de automatización hasta una Inteligencia Artificial, está construida sobre cuatro bloques lógicos elementales.

☀ Metáfora Central: El Asistente, las Cajas, el Semáforo y la Licuadora

Imagina que la computadora es un asistente súper eficiente pero literal: las variables son cajas etiquetadas donde guarda cosas, el `if` es un semáforo que decide el camino según la luz, el `for` es una cinta transportadora que procesa elementos uno a uno, y la función `def` es una licuadora que recibe ingredientes y entrega un licuado.

Principios Teóricos y Modelo Mental

1. Variables (Memoria): Espacios con nombre para retener datos temporalmente. 2. Condicionales (Decisión): Bifurcaciones lógicas según condiciones booleanas. 3. Bucles (Repetición): Automatización de tareas repetitivas sin duplicar código. 4. Funciones (Modularidad): Bloques reutilizables con entradas y salidas bien definidas.

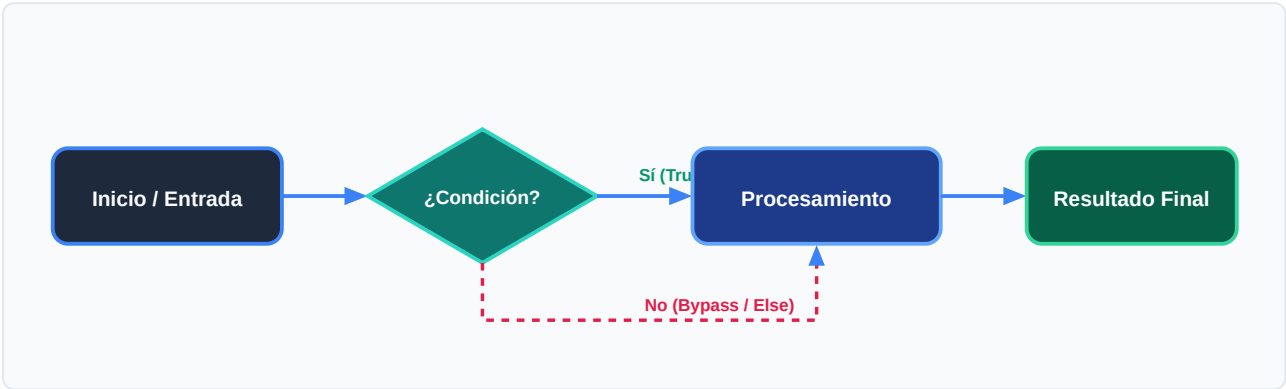
La magia del software no radica en la complejidad de cada pieza aislada, sino en la sinergia con la que se combinan para modelar la realidad.

⚡ Regla de Oro en Python

Python es un lenguaje interpretado, de tipado dinámico y fuertemente tipado: respeta la indentación y la semántica.

Diagrama de Ejecución Secuencial y Control

Cómo el intérprete de Python procesa el código línea por línea desde el punto de entrada hasta la resolución.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Lee la instrucción inicial e inicializa el entorno de variables en memoria.	Tabla de símbolos vacía -> asigna valores
2. Evaluación	Evalúa expresiones booleanas en condicionales para determinar la ruta.	Evalúa True o False en CPU
3. Transformación	Ejecuta el bloque indentado correspondiente a la condición satisfecha.	Transformación de variables
4. Retorno / Salida	Invoca funciones y devuelve el resultado a la consola con print().	Liberación de stack frame

Visualización Mental

Piensa en el intérprete de Python como un lector con un marcador que avanza de arriba a abajo, saltando sólo cuando encuentra estructuras de control.

Script Integrador de los 4 Pilares

main.py (Python 3.10+)

PEP 8 Compliant

```
# 1. Definición de Función Reutilizable (La Licuadora)
def evaluar_estudiante(nombre: str, nota: float) -> str:
    if nota >= 7.0:
        return f"¡Felicidades {nombre}! Aprobaste con éxito 🚀"
    else:
        return f"Ánimo {nombre}, debes reforzar los conceptos 📖"

# 2. Variables y Colección (Cajas en memoria)
estudiantes = ["Ana", "Carlos", "Sofía"]
calificaciones = [9.5, 5.8, 8.2]

# 3. Bucle de Procesamiento (Cinta Transportadora)
for i in range(len(estudiantes)):
    resultado = evaluar_estudiante(estudiantes[i], calificaciones[i])
    print(resultado)
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar los dos puntos (:) al final de las estructuras if, for o def, o mezclar espacios y tabulaciones en la indentación.

✗ Antipatrón

```
if nota > 5
print("Aprobado") # Error de sintaxis
```

✓ Patrón Correcto

```
if nota > 5:
    print("Aprobado") # Correcto e indentado
```

Clase 02: Variables y Tipos de Datos

Una variable es un identificador que apunta a una ubicación de memoria donde reside un valor con un tipo de dato específico.

☀ Metáfora Central: El Almacén, el Collar de Letras y el Micrófono

Imagina un almacén con cajas etiquetadas. Una caja pequeña guarda números enteros (int), una caja de precisión con decimales guarda números reales (float), una caja larga guarda un collar de letras enhebradas (str) y un interruptor de encendido/apagado representa un valor booleano (bool).

Principios Teóricos y Modelo Mental

Python utiliza tipado dinámico: no necesitas declarar el tipo de antemano, el intérprete lo infiere en tiempo de asignación.

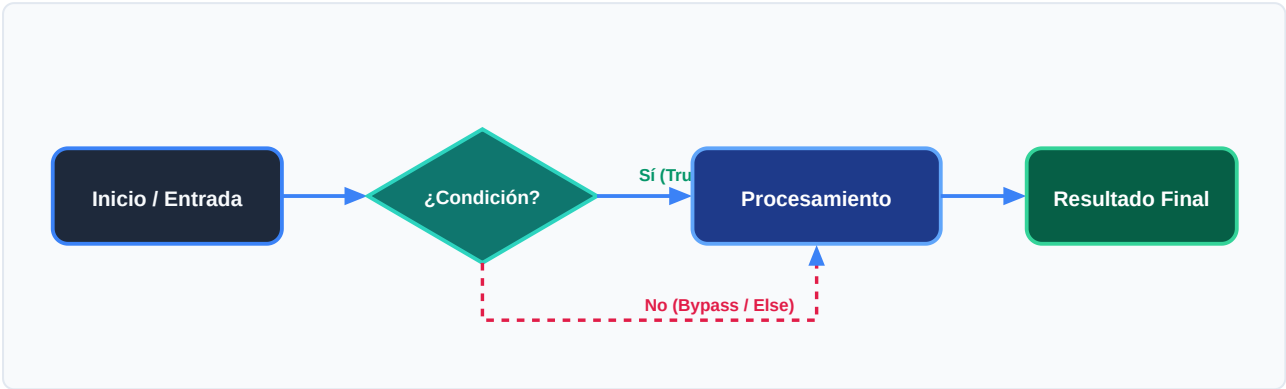
La función `input()` SIEMPRE devuelve una cadena de texto (str). Para operar matemáticamente con ella es imperativo hacer casting mediante `int()` o `float()`.

⚡ Regla de Oro en Python

Nunca sumes texto con números sin convertir; `'10' + 5` genera `TypeError`, pero `int('10') + 5` produce 15.

Ciclo de Conversión y Entrada de Datos (Casting)

Flujo de recepción de datos por teclado, validación de tipo y operación aritmética en memoria.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	La función <code>input()</code> captura la entrada del teclado como string.	Buffer de entrada -> '25' (str)
2. Evaluación	La función <code>int()</code> o <code>float()</code> transforma los caracteres en número binario.	Casting -> 25 (int)
3. Transformación	La ALU del procesador realiza la operación matemática solicitada.	$25 * 2 = 50$ en CPU
4. Retorno / Salida	f-string formatea el resultado y lo proyecta en la salida estándar.	Render en pantalla

Visualización Mental

Siempre valida y castea los datos en la frontera de entrada del programa antes de procesarlos en la lógica de negocio.

Calculadora de Ahorro con Conversión de Tipos

main.py (Python 3.10+)

PEP 8 Compliant

```
# Entrada de datos con conversión directa
nombre_usuario: str = input("Ingresa tu nombre: ")
ingreso_mensual: float = float(input("Ingreso mensual ($): "))
porcentaje_ahorro: float = float(input("Porcentaje a ahorrar (%): "))

# Cálculo matemático
monto_ahorro: float = ingreso_mensual * (porcentaje_ahorro / 100.0)
es_meta_alta: bool = monto_ahorro >= 500.0

# Salida formateada con f-strings
print(f"\n--- Reporte Financiero de {nombre_usuario} ---")
print(f"Ahorro estimado: ${monto_ahorro:,.2f}")
print(f"¿Es un ahorro significativo?: {es_meta_alta}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Intentar convertir una cadena con caracteres alfabéticos a int (ej: int('hola')), lo cual dispara un ValueError.

✗ Antipatrón

```
edad = input('Edad: ')
total = edad + 5 # TypeError: str + int
```

✓ Patrón Correcto

```
edad = int(input('Edad: '))
total = edad + 5 # Correcto: suma entera
```

Clase 03: Control de Flujo - Condicionales

Un programa no es una línea recta; es un camino con encrucijadas donde el flujo toma una dirección según las condiciones.

☀ Metáfora Central: El Guardia de la Puerta y el Menú de Opciones

Imagina un guardia en la entrada de un club: revisa tu entrada (if). Si tienes pase VIP entra gratis (if), si tienes entrada general paga boleto (elif), y si no tienes entrada se le deniega el acceso (else).

Principios Teóricos y Modelo Mental

Operadores relacionales: == (igualdad), != (diferente), > (mayor), < (menor), >= (mayor o igual), <= (menor o igual).

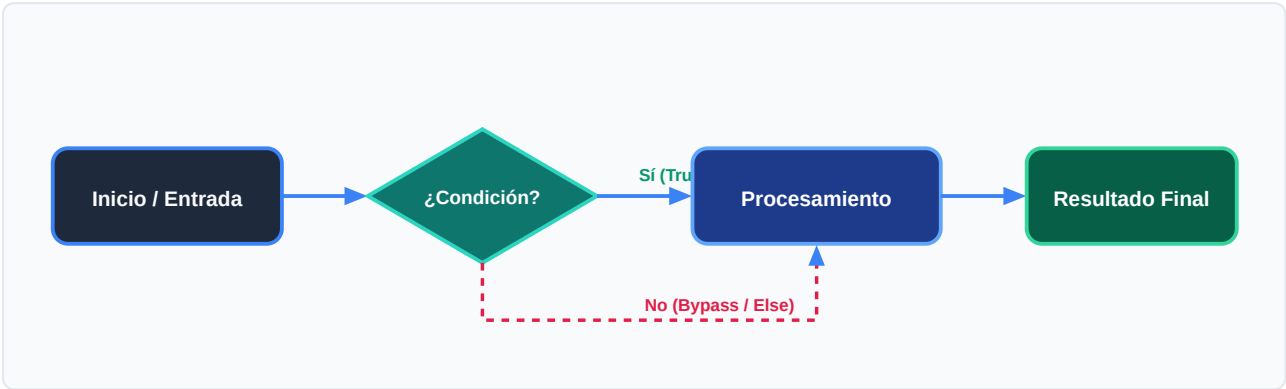
Operadores lógicos: and (ambas condiciones deben ser True), or (al menos una True), not (invierte el valor de verdad).

⚡ Regla de Oro en Python

En una cadena if-elif-else, tan pronto como una condición resulta True, se ejecuta su bloque y se omiten todas las demás.

Árbol de Decisión y Evaluación de Condiciones

Representación del flujo booleano con múltiples alternativas excluyentes.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Evalúa la primera condición del if principal.	Condición 1: ¿edad >= 18?
2. Evaluación	Si es True, entra al bloque if y salta al final de la estructura.	Ejecuta bloque prioritario
3. Transformación	Si es False, evalúa secuencialmente los bloques elif.	Condición 2: ¿tiene_permiso?
4. Retorno / Salida	Si ninguna condición previa fue True, se ejecuta el bloque else por defecto.	Rama fallback de seguridad



Visualización Mental

Ordena tus condiciones de la más específica a la más general para evitar que un caso amplio oculte casos particulares.

Sistema de Clasificación de Préstamos Bancarios

main.py (Python 3.10+)

PEP 8 Compliant

```
salario = float(input("Salario mensual ($): "))
puntaje_credito = int(input("Puntaje crediticio (300-850): "))
tiene_deudas = input("¿Tiene deudas activas? (s/n): ").lower() == "s"

if salario >= 3000.0 and puntaje_credito >= 720 and not tiene_deudas:
    estado = "Aprobado Premium (Tasa de interés preferencial)"
elif salario >= 1800.0 and puntaje_credito >= 650:
    estado = "Aprobado Estándar (Sujeto a verificación)"
elif salario >= 1200.0 or puntaje_credito >= 600:
    estado = "Requiere Codeudor o Aval"
else:
    estado = "Rechazado (No cumple los requisitos mínimos)"

print(f"\nResultado de la solicitud: {estado}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Confundir el operador de asignación (=) con el operador de comparación (==).

✗ Antipatrón

```
if rol = "admin": # SyntaxError
    print("Acceso total")
```

✓ Patrón Correcto

```
if rol == "admin": # Comparación correcta
    print("Acceso total")
```

Clase 04: Control de Flujo - Bucles

La mayor fortaleza de una computadora es su capacidad para ejecutar una misma tarea millones de veces sin cansarse ni cometer errores.

☀ Metáfora Central: Las Vueltas a la Pista y el Termostato

El bucle for es como un atleta que da un número exacto de vueltas a la pista de carreras (5 vueltas definidas). El bucle while es como el termostato de un calentador: funciona continuamente mientras la temperatura esté por debajo de 22 grados, y se detiene automáticamente cuando se alcanza la meta.

Principios Teóricos y Modelo Mental

Bucle for: Ideal cuando conoces de antemano el número de repeticiones o cuando recorres una colección finita.

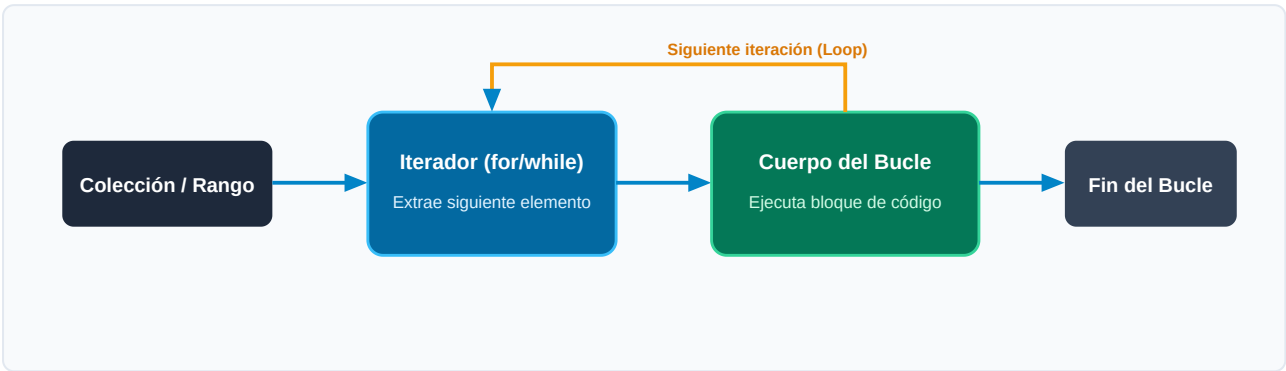
Bucle while: Ideal cuando la repetición depende de una condición externa que puede cambiar dinámicamente durante la ejecución.

⚡ Regla de Oro en Python

Todo bucle while debe modificar en su cuerpo la variable de control; de lo contrario, se convierte en un bucle infinito que congela el programa.

Ciclo de Vida de una Iteración

Estructura del flujo de control iterativo y mecanismos de interrupción anticipada.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Inicializa el índice o evalúa la condición de entrada del bucle.	Variable de control lista
2. Evaluación	Ejecuta las instrucciones del bloque interno.	Cálculo en la iteración actual
3. Transformación	Si encuentra 'continue', salta directamente a la siguiente iteración.	Bypass de código restante
4. Retorno / Salida	Si encuentra 'break', aborta el bucle inmediatamente hacia la siguiente línea externa.	Salida forzada del ciclo

Visualización Mental

Visualiza el bucle como una rueda que gira; cada vuelta procesa un dato individual hasta que se agota el combustible de la condición.

Sistema de Autenticación con Reintentos Limitados

main.py (Python 3.10+)

PEP 8 Compliant

```
PASSWORD_SECRETA = "python2026"
intentos_maximos = 3
intentos_realizados = 0
acceso_concedido = False

while intentos_realizados < intentos_maximos:
    intento = input(f"Intento [{intentos_realizados + 1}/{intentos_maximos}] - Contraseña: ")
    if intento == PASSWORD_SECRETA:
        acceso_concedido = True
        print("¡Acceso exitoso al sistema! 🗝️")
        break
    else:
        print("❌ Contraseña incorrecta.")
        intentos_realizados += 1

if not acceso_concedido:
    print("🚫 Sistema bloqueado por demasiados intentos fallidos.")
```

⚠️ Gotcha Frecuente (Trampa de Principiante)

Olvidar incrementar el contador en un bucle while, resultando en un bucle infinito que consume el 100% de la CPU.

❌ Antipatrón

```
i = 0
while i < 5:
    print(i) # Olvido de i += 1 -> Bucle infinito
```

✅ Patrón Correcto

```
for i in range(5):
    print(i) # Seguro, limpio e idiomático
```

Clase 05: Listas y Colecciones de Datos

En el mundo real rara vez trabajamos con datos aislados; casi siempre gestionamos conjuntos de elementos como listas de clientes, precios o mediciones.

☀ **Metáfora Central: La Mochila del Programador y los Casilleros**

Imagina una fila de casilleros escolares numerados desde el 0. En cada casillero puedes guardar lo que quieras. Las listas son casilleros que puedes abrir, cambiar y reordenar (mutables). Las tuplas son cajas de cristal selladas: puedes ver lo que hay dentro, pero nadie puede alterarlo (inmutables).

Principios Teóricos y Modelo Mental

Indexación: El primer elemento está en el índice 0, y el último en el índice -1.

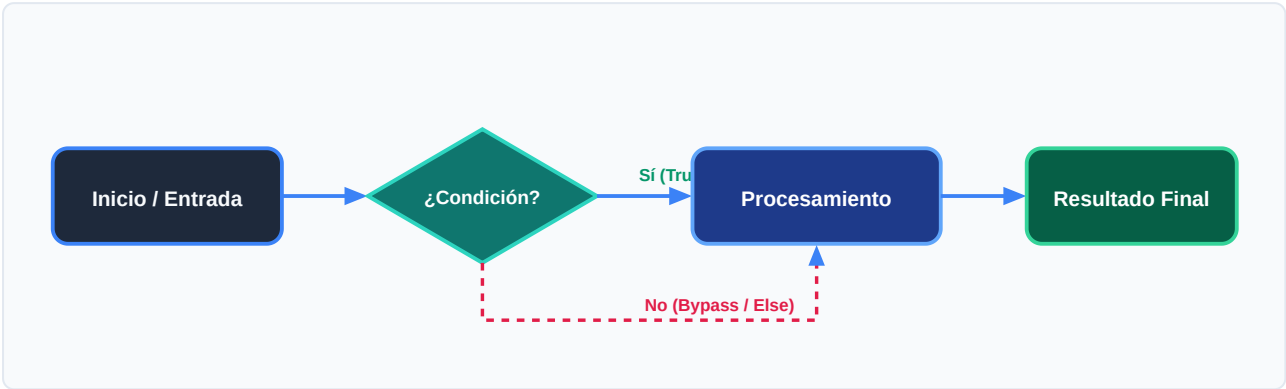
Slicing: La sintaxis `lista[inicio:fin:paso]` permite extraer subconjuntos sin modificar la lista original.

⚡ **Regla de Oro en Python**

Las listas son mutables (se modifican en el mismo lugar de memoria); las tuplas son inmutables y ofrecen mayor seguridad e integridad.

Anatomía de la Indexación y Operaciones de Slicing

Mapeo de memoria para índices directos, inversos y sub-rangos de datos.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Python asigna un puntero de memoria ordenado a cada elemento.	['A', 'B', 'C', 'D']
2. Evaluación	Índices positivos: [0]=A, [1]=B, [2]=C, [3]=D.	Lectura hacia adelante
3. Transformación	Índices negativos: [-1]=D, [-2]=C, [-3]=B, [-4]=A.	Lectura desde el final
4. Retorno / Salida	Slicing [1:3] extrae los índices 1 y 2 (el límite superior es excluyente).	Nueva lista: ['B', 'C']

Visualización Mental

Recuerda siempre la regla del límite superior: lista[0:3] extrae 3 elementos (índices 0, 1 y 2), el 3 queda fuera.

Gestión de Carrito de Compras con Listas

main.py (Python 3.10+)

PEP 8 Compliant

```
carrito: list[str] = ["Laptop", "Mouse", "Teclado"]

# 1. Agregar elementos
carrito.append("Monitor 4K")
carrito.insert(1, "Auriculares")

# 2. Slicing (primeros 3 productos)
prioritarios = carrito[0:3]
print(f"Productos prioritarios: {prioritarios}")

# 3. Eliminar y extraer
eliminado = carrito.pop()
print(f"Producto extraído: {eliminado}")

# 4. Iteración elegante con enumeración
for idx, prod in enumerate(carrito, start=1):
    print(f"{idx}. {prod}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Copiar una lista por asignación simple (lista2 = lista1) solo copia la referencia, no los datos.

✗ Antipatrón

```
a = [1, 2, 3]
b = a
b.append(4) # ¡Modifica también la lista 'a'!
```

✓ Patrón Correcto

```
a = [1, 2, 3]
b = a.copy() # Copia superficial independiente
b.append(4)
```

Clase 06: Diccionarios y Mapeos Clave-Valor

Buscar un dato por su posición (índice 4) es poco intuitivo; en el mundo real buscamos por nombre, correo o ID.

☀ Metáfora Central: La Agenda Telefónica y el Expediente Médico

Un diccionario es como tu agenda del teléfono: no buscas a tu mamá por el número de orden en que la agregaste, buscas la etiqueta 'Mamá' (la clave) y obtienes su número de teléfono (el valor).

Principios Teóricos y Modelo Mental

Las claves en un diccionario deben ser únicas e inmutables (comúnmente strings o ints). Los valores pueden ser de cualquier tipo, incluidas listas u otros diccionarios.

La búsqueda en un diccionario es instantánea (tiempo constante $O(1)$) gracias al algoritmo interno de tabla hash.

⚡ Regla de Oro en Python

Nunca accedas a una clave con `dict['clave']` si no estás 100% seguro de que existe; usa `dict.get('clave', valor_por_defecto)` para evitar `KeyError`.

Arquitectura de un Diccionario y Búsqueda por Hash

Cómo Python mapea claves alfanuméricas a ubicaciones de memoria específicas.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Python aplica una función hash a la clave (ej: <code>hash('email')</code>).	Clave -> Hash ID numérico
2. Evaluación	Localiza el casillero exacto en la tabla hash de memoria.	Búsqueda $O(1)$
3. Transformación	Recupera o modifica el valor asociado sin recorrer toda la estructura.	Lectura/Escritura inmediata
4. Retorno / Salida	Permite serialización directa hacia y desde formato JSON para APIs web.	Compatibilidad universal

Visualización Mental

Los diccionarios son el equivalente en Python a los objetos de JavaScript o los registros de bases de datos NoSQL.

Sistema de Gestión de Inventario con Diccionarios

main.py (Python 3.10+)

PEP 8 Compliant

```
inventario = {
    "PROD-001": {"nombre": "Teclado Mecánico", "precio": 85.0, "stock": 12},
    "PROD-002": {"nombre": "Mouse Ergonómico", "precio": 45.0, "stock": 0}
}

# Acceso seguro con .get()
sku_buscado = "PROD-001"
producto = inventario.get(sku_buscado, None)

if producto:
    print(f"Producto: {producto['nombre']} | Stock: {producto['stock']} uds")

# Iteración completa de claves y valores
for sku, datos in inventario.items():
    disponible = "En Stock" if datos["stock"] > 0 else "Agotado"
    print(f"[{sku}] {datos['nombre']} -> {disponible}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Consultar una clave inexistente con corchetes (dict['inexistente']) provoca un KeyError que detiene el programa.

✗ Antipatrón

```
user = {'nombre': 'Leo'}
print(user['edad']) # KeyError: 'edad'
```

✓ Patrón Correcto

```
user = {'nombre': 'Leo'}
print(user.get('edad', 0)) # Retorna 0 de forma segura
```

Clase 07: Funciones Reutilizables y Modulares

El código profesional no se escribe dos veces; cuando una lógica se necesita en múltiples lugares, se encapsula en una función.

☀ Metáfora Central: El Electrodoméstico y la Entrega del Cajero

Una función es como un electrodoméstico: tiene una ranura de entrada (parámetros), un motor interno que realiza una tarea específica, y una bandeja de salida donde entrega el resultado terminado (return).

Principios Teóricos y Modelo Mental

Parámetros vs Argumentos: Los parámetros son los nombres en la firma (def), los argumentos son los valores reales que pasas al invocarla.

Diferencia crucial: print() solo muestra texto en la pantalla pero devuelve None; return devuelve el valor a la variable que llamó a la función para seguir trabajando con él.

⚡ Regla de Oro en Python

Una función debe hacer una sola cosa y hacerla excepcionalmente bien (Principio de Responsabilidad Única).

Caja Negra Funcional y Ámbito de Variables (Scope)

Flujo de invocación, paso de argumentos, aislamiento de variables locales y retorno de valor.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	La función es llamada y se asignan los argumentos a los parámetros.	Creación del Call Stack Frame
2. Evaluación	Se ejecutan las instrucciones en un ámbito local aislado.	Variables locales temporales
3. Transformación	La instrucción 'return' finaliza la ejecución de la función y emite el resultado.	Envío del valor de retorno
4. Retorno / Salida	El stack frame se destruye y la memoria local se libera.	Retorno al flujo principal



Visualización Mental

Las variables creadas dentro de una función mueren cuando la función termina: nunca intentes acceder a una variable local desde fuera.

Módulo de Facturación con Funciones Tipadas

main.py (Python 3.10+)

PEP 8 Compliant

```
def calcular_total_factura(
    subtotal: float,
    tasa_impuesto: float = 0.21,
    descuento: float = 0.0
) -> dict[str, float]:
    """Calcula el desglose final de una factura comercial."""
    monto_descuento = subtotal * descuento
    base_imponible = subtotal - monto_descuento
    impuestos = base_imponible * tasa_impuesto
    total_pagar = base_imponible + impuestos

    return {
        "subtotal": subtotal,
        "descuento_aplicado": monto_descuento,
        "impuestos": impuestos,
        "total": round(total_pagar, 2)
    }

# Uso con argumentos por nombre (keyword arguments)
factura = calcular_total_factura(subtotal=150.0, descuento=0.10)
print(f"Total a pagar: ${factura['total']}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar argumentos mutables por defecto (como `def func(lista=[])`); la lista se comparte entre llamadas sucesivas.

✗ Antipatrón

```
def agregar(item, lista=[]): # ¡Peligro mutable!
    lista.append(item)
    return lista
```

✓ Patrón Correcto

```
def agregar(item, lista=None):
    if lista is None: lista = []
    lista.append(item)
    return lista
```

Clase 08: Integración Total & Proyecto Integrador

Llegó el momento de unir todas las piezas: variables, condicionales, bucles, listas, diccionarios y funciones trabajando en armonía.

☀ **Metáfora Central: El Casco de Seguridad y Salir a Rodar en Bici**

Hasta ahora hemos practicado el equilibrio con las rueditas de entrenamiento. Hoy nos quitamos las rueditas, nos ponemos el casco de seguridad y salimos a rodar en la bicicleta por nosotros mismos en el mundo real.

Principios Teóricos y Modelo Mental

Patrón de Menú Principal: Un bucle infinito `while True` mantiene viva la aplicación hasta que el usuario decida salir explícitamente.

Capa de Datos: Una lista de diccionarios en memoria actúa como la base de datos temporal de la aplicación.

⚡ **Regla de Oro en Python**

Separa la presentación (`print`, `input`) de la lógica de negocio (las funciones que agregan, buscan y transforman datos).

Diagrama de Arquitectura de la Aplicación CLI

Interacción entre la capa de interfaz de consola, el enrutador de comandos y el modelo de datos.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Bucle principal muestra el menú de opciones (1. Agregar, 2. Listar, 3. Completar, 4. Salir).	Esperando opción del usuario
2. Evaluación	Enrutador if/elif invoca la función específica según la opción elegida.	Despacho a función modular
3. Transformación	La función ejecuta la operación CRUD sobre la lista de tareas en memoria.	Actualización del estado
4. Retorno / Salida	Se muestra retroalimentación visual al usuario y se reinicia el ciclo del menú.	Ciclo listo para nueva orden

Visualización Mental

Esta arquitectura modular en consola es idéntica en concepto a los controladores y servicios de las APIs web modernas.

Implementación del Task Manager CLI

main.py (Python 3.10+)

PEP 8 Compliant

```
tareas: list[dict] = []

def agregar_tarea(titulo: str) -> None:
    nueva_tarea = {"id": len(tareas) + 1, "titulo": titulo, "completada": False}
    tareas.append(nueva_tarea)
    print(f"✅ Tarea #{nueva_tarea['id']} agregada con éxito.")

def listar_tareas() -> None:
    if not tareas:
        print("📭 No hay tareas registradas.")
        return
    for t in tareas:
        estado = "✅ [LISTA]" if t["completada"] else "⌚ [PENDIENTE]"
        print(f"#{t['id']} - {t['titulo']} {estado}")

def completar_tarea(id_tarea: int) -> None:
    for t in tareas:
        if t["id"] == id_tarea:
            t["completada"] = True
            print(f"🚀 Tarea #{id_tarea} marcada como completada.")
            return
    print("❌ ID de tarea no encontrado.")
```

⚠️ Gotcha Frecuente (Trampa de Principiante)

Escribir código espagueti con cientos de líneas sin funciones y mezclando variables globales descontroladas.

❌ Antipatrón

```
# Código monolítico sin funciones ni modularidad
while True:
    op = input()
    # 300 líneas de if/else anidados sin separación
```

✅ Patrón Correcto

```
# Código desacoplado
def main():
    while True:
        mostrar_menu()
        procesar_opcion()
```

Conclusiones Generales del Curso

Has completado el recorrido integral de **Curso 1: Fundamentos Básicos de Python**. El dominio de estos conceptos te sitúa con una ventaja competitiva sólida para el desarrollo de software profesional e Inteligencia Artificial.

Hitos Alcanzados

Comprensión profunda de la arquitectura, asimilación de patrones de diseño idiomáticos en Python, resolución de problemas algorítmicos y capacidad de integración de proyectos reales.

Fuentes Bibliográficas Canónicas

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica del lenguaje	docs.python.org/3/
PEP 8 - Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones profesionales de ingeniería	realpython.com
Suite wisrovi en GitHub	Librerías open source de alto rendimiento	github.com/wisrovi

Agradecimiento del Instructor

Gracias por tu disciplina, curiosidad y compromiso en cada lección. Continúa pedaleando y construyendo software con propósito. ¡Nos vemos en el siguiente nivel!